

Programación III — Guía de Trabajos Prácticos

Tema: Análisis de Complejidad en Algoritmos Recursivos

Cátedra: Programación III

Modalidad: Resolución en Clase

Alumno/a:

Fecha: ____ / ____ / ____

Ejercicio 1: Método de Sustracción — Torres de Hanói

Dado el algoritmo clásico para resolver el problema de las Torres de Hanói:

```
public void hanoi(int n, char origen, char auxiliar, char destino) {  
    if (n == 1) {  
        System.out.println("Mover disco 1 de " + origen + " a " + destino);  
    } else {  
        hanoi(n - 1, origen, destino, auxiliar);  
        System.out.println("Mover disco " + n + " de " + origen + " a " + destino);  
        hanoi(n - 1, auxiliar, origen, destino);  
    }  
}
```

Consignas:

1. Determinar los valores de los parámetros a , b y k .
2. Plantear la ecuación de recurrencia $T(n)$.
3. Clasificar la complejidad asintótica Θ aplicando la regla correspondiente del método de sustracción.

Espacio de Resolución:

Ejercicio 2: Método de División — Búsqueda Binaria

Dada la implementación de la búsqueda binaria recursiva sobre un arreglo ordenado:

```
public int busquedaBinaria(int[] v, int inicio, int fin, int x) {
    if (inicio > fin) return -1;

    int medio = (inicio + fin) / 2;
    if (v[medio] == x) return medio;

    if (v[medio] > x) {
        return busquedaBinaria(v, inicio, medio - 1, x);
    } else {
        return busquedaBinaria(v, medio + 1, fin, x);
    }
}
```

Consignas:

1. Identificar los valores de a , b y k en función del tamaño del subproblema $n = fin - inicio + 1$.
2. Evaluar la relación comparativa entre a y b^k .
3. Determinar el orden de complejidad asintótica Θ aplicando el método de división.

Espacio de Resolución:

Ejercicio 3: Análisis con Trabajo Extra ($k \neq 0$)

Considere la siguiente función recursiva que procesa subarreglos:

```
public void procesarSubarreglos(int[] v, int inicio, int fin) {
    int n = fin - inicio + 1;
    if (n <= 1) return;

    // Trabajo extra fuera de las llamadas recursivas
    for (int i = inicio; i <= fin; i++) {
        System.out.print(v[i] + " ");
    }
    System.out.println();

    int medio = (inicio + fin) / 2;
    procesarSubarreglos(v, inicio, medio);
    procesarSubarreglos(v, medio + 1, fin);
}
```

Consignas:

1. Determinar el grado k del polinomio correspondiente al trabajo realizado fuera de las llamadas recursivas.
2. Identificar los parámetros a y b .
3. Calcular b^k , realizar la comparación correspondiente y determinar la complejidad Θ .

Espacio de Resolución:

Ejercicio 4: Razonamiento e Identificación (V/F)

Indique si las siguientes afirmaciones son **Verdaderas (V)** o **Falsas (F)**. Justifique conceptualmente cada respuesta empleando las reglas aprendidas:

Afirmación	V / F
1. En el método de sustracción, si realizamos 2 llamadas recursivas ($a = 2$) restando 1 unidad a la entrada ($b = 1$), la complejidad resultante siempre será de orden lineal $\Theta(n)$.	[]
2. Si en un algoritmo del método de división reducimos la entrada a la mitad ($b = 2$) y realizamos 3 llamadas recursivas ($a = 3$) con costo externo constante ($k = 0$), el algoritmo es de orden polinómico $\Theta(n^{\log_2 3})$.	[]

Justificaciones: